

Le bug des calories, expliqué simplement

Force Tracker — corrigé en ft-v1177, le 8 septembre 2026

1. Ce qui se passait

Quand tu reprenais un aliment depuis « **Mes aliments** » (Nutrition > Journal > « À la main »), l'application **gardait les calories** de la fois d'avant mais **oubliait la quantité** à laquelle elles correspondaient.

Ta ratatouille était notée **380 g = 274 kcal**. Tu la reprenais, tu mets **110 g** : l'application affichait encore **274 kcal**. Elle venait de décider, toute seule, que *274 kcal pèsent 110 g*.

Et c'est là que ça devient grave : elle a **enregistré cette conclusion**. Elle en a déduit que ta ratatouille fait **249 kcal pour 100 g**, au lieu de 72. À partir de là, tout ce qu'elle calculait était faux — mais *cohérent*, donc invisible.

C'est pour ça que tu voyais des chiffres délirants sans comprendre d'où ils venaient : l'erreur ne se voyait pas au moment où elle se produisait, elle se voyait **les jours suivants**.

2. Pourquoi ça ne le faisait pas à chaque fois

Il y a **trois façons** de reprendre un aliment dans l'app, et elles ne passaient pas par le même code. **Une seule des trois était cassée** — celle que tu utilises le plus.

Comment tu reprends l'aliment	État avant ft-v1177
Tu tapes son nom dans « Ce que tu as mangé »	[OK] correct — 110 g donnait bien 79 kcal
Tu tapes son nom dans la liste « Mes aliments »	[X] cassé — 110 g donnait 274 kcal
Tu appuies sur « + Ajouter » dans cette liste	[X] cassé — enregistrait sans aucune quantité

Le correctif était déjà écrit dans l'application, sur le premier chemin. Il manquait simplement sur les deux autres. Deux lignes de code.

3. Ce qui est réparé, et ce qui ne l'est pas

	État
Les 3 chemins de reprise gardent la quantité	[OK] fait — en ligne depuis ft-v1177
Reprendre un aliment ouvre le champ sur ta dernière quantité	[OK] fait
Les lignes déjà fausses de ton journal	[X] pas touchées — voir ci-dessous
Les deux blocs Quantité affichés en même temps	[X] version suivante

Pourquoi on ne répare pas ton historique tout de suite. Certaines lignes ont été enregistrées **sans aucune quantité**. Devant « 323 kcal », rien ne permet de savoir si c'était 100 g ou 300 g de steak. Deviner produirait des chiffres faux qui auraient l'air justes — exactement le problème qu'on vient de corriger.

L'outil de récupération viendra après, avec **trois niveaux honnêtes** : ce qui est certain (recalculé tout seul), ce qui est ambigu (on te demande), et ce qui est perdu (on le dit).

4. Ce que tu peux vérifier toi-même

Ferme et rouvre l'application. Puis : **Nutrition > Journal > « À la main »**, reprends ta ratatouille dans « Mes aliments », mets **110**.

Tu dois voir **79 kcal**. Et le champ doit s'ouvrir sur **380** (ta dernière quantité) au lieu d'être vide.

Dossier technique — pour le relecteur externe

Réponse a l'audit du 8 septembre 2026 — ce qui a ete reproduit, corrigé, et laissé ouvert

1. Ce que l'audit avait raison de dire

Sa conclusion principale est adoptée telle quelle. J'avais écrit « le reste du code est juste » ; son §7 refuse la formule, a raison. L'existence de per100, de q/u et d'un _qtyRescale déterministe ne prouve rien sur la transmission.

La formulation retenue est la sienne : le modèle de données existe et est exploitable, mais ses invariants n'étaient pas protégés sur tous les chemins de reprise.

Son cas témoin (ratatouille) était le bon, sa cause racine était la bonne, et sa consigne de ne pas toucher a _qtyRescale était juste : il calcule correctement a partir d'une référence déjà corrompue en amont.

2. Reproduction — mesurée avant toute correction

Rejoué dans un vrai navigateur (Chromium + Playwright), par les fonctions de production, avec l'entrée exacte de son export. **Les trois états correspondent au chiffre pres.**

Etape	Son export réel	Ma reproduction
31/08 — entrée de départ	380 g = 274 kcal / 4 P / 23 G / 15 L	(fixture identique)
01/09 — reprise, on demande 110 g	274 kcal / 4 / 23 / 15	274 / 4 / 23 / 15
référence dérivée et enregistrée	~249 kcal / 100 g	249
02/09 — reprise suivante a 180 g	448 / 7 / 38 / 25	448 / 7 / 38 / 25

3. La cause racine, et le discriminant

Deux chemins de reprise, **la même entrée** sans per100 (380 g = 274 kcal) :

Fonction	Référence construite	110 g rendait
_afSuggPrendreLocale (recherche par nom)	{base:274, q:380, u:'g'}	79 kcal
quickFillFood (« Mes aliments »)	{base:274, q:1, u:''}	274 kcal

Les deux lignes qui préservent le couple existaient déjà, dans la fonction voisine. Elles étaient posées sur **une seule des deux portes**. C'est la sixième occurrence documentée de ce motif dans ce fichier (règle interne R8, « la jumelle »).

La chaîne complète, une fois la quantité perdue :

1. Sans per100, it.q est la **seule** chose qui relie 274 kcal au monde réel — la branche else la jétait.
2. _afMajAncre traitait alors l'ecran comme « une portion » (q:1). Déclarer 110 g revenait a affirmer que ces 274 kcal pèsent 110 g.
3. _provFood **fabrique** ensuite un pour-100 g depuis cet appariement ($274 / 110 \times 100 = 249$) et **l'enregistre avec l'aliment**.

Le point 3 n'est pas un bug : c'est le calibrage par le poids, correct pour un aliment neuf. Il ne fait que propager une erreur née en amont. C'est la moitié que l'audit n'avait pas identifiée, et c'est elle qui rend le dégât permanent.

4. Ce qui a ete corrigé (ft-v1177)

#	Fonction	Correction
1	quickFillFood	Ancre la référence sur <code>it.q</code> quand il n'y a pas de <code>per100</code> — les deux lignes de la fonction voisine, portées ici. Uniquement en grammes (une quantité en <code>ml</code> n'ancre rien : pas de densité inventée).
2	quickAddFood	Transmet <code>q</code> , <code>u</code> , <code>per100</code> , <code>sourceId</code> , état au lieu des seuls totaux. Plus de ligne non convertible.
3	<code>_provFood</code>	<code>q/u</code> traversent enfin la liste blanche. Troisième fois que ce même oubli se produit dans cette fonction, qui porte déjà deux avertissements en majuscules le disant.

Non touchés, a la demande explicite de l'audit :

- `_qtyRescale` — il calcule juste depuis une référence corrompue en amont ; le modifier compenserait au lieu de corriger.
- `_afSuggPrendreLocale` — son §8-3 demandait de la vérifier : **elle était déjà correcte**. Mesuré, puis laissé intact.

5. Ses dix tests de non-régression

Ecrits tels qu'il les a demandés, avec ses valeurs interdites. **16 témoins, tous verts**, intégrés au banc d'essai permanent (bloc CCLXXV).

Test	Attendu	Interdit	Obtenu
1 — ratatouille, 110 g	~79 kcal	274	79
2 — propagation, 180 g	~130 kcal	448	130
3 — <code>per100</code> immuable	72 / 100 g	dérive	72 sur 100/380/110/180 g
4 — sans <code>per100</code>	274 x 110/380	274 x 110/110	79
5 — <code>quickAddFood</code>	<code>q</code> , <code>u</code> , <code>per100</code> gardés	null	380 g, 72
6 — steak U, 250 -> 300 g	~388 kcal	~194	388
7 — Iso Zero 20/40/50 g	proportionnel	35 P partout	78/156/195
9 — modif d'une entrée	<code>per100</code> et <code>source</code> intacts	réécriture	intacts
10 — 10 cycles	aucune dérive	dérive cumulative	144 kcal x 10

Le test 8 (exclusivité des deux blocs Quantité) part avec la version suivante : l'audit dit lui-même que c'est un bug distinct.

6. Contrôle négatif — ce qui a failli passer

Six mutations, chacune cassant un point précis. **Toutes font rougir un témoin**, ce qui prouve que les témoins mesurent quelque chose.

Mutation	Témoins rouges
L'arbre d'avant la correction	13
<code>quickFillFood</code> rejette à nouveau <code>it.q</code>	11 — le défaut d'origine
<code>quickAddFood</code> ne transmet plus rien	2
La liste blanche de <code>_provFood</code>	2 (les mêmes — deux faces d'une garantie)
Mon premier correctif, remis	1 — exactement le témoin de l'aller-retour
L'ancrage accepterait les <code>ml</code>	1

La mutation la plus utile est la cinquième, parce qu'elle rejoue mon erreur.

Mon premier correctif marquait le poids repris comme « déclaré par la personne ». Le mini-banc était a 14/14. **Mesure sur un aller-retour d'unité : déclarer 110 g redonnait 274 kcal** — le défaut même que la version corrige.

Le drapeau signifie « *la personne a déclaré un poids pour ce qui est AFFICHÉ* ». Un poids **hérité** d'une entrée enregistrée n'est pas cela : dès qu'elle change la quantité, la référence ne correspond plus à ce qu'elle voit. **La différence n'est pas le geste, c'est QUI a pose le poids.**

7. Ce qui reste ouvert — et où son avis est utile

L'historique n'est **pas** réparé, conformément à son §11. Trois questions restent, et ce sont des **décisions produites**, pas des questions techniques :

Question	Ce qui est déjà tranché
1. Quelles règles définissent un « MATCH CERTAIN » recalculable automatiquement ? Suffit-il qu'une autre ligne du même aliment porte un per100 crédible ?	Rien. Aucune approximation silencieuse ne sera faite.
2. Faut-il marquer visiblement dans le journal les lignes non convertibles, ou les laisser telles quelles ?	Le principe interne dit : informer sans décider, et ne jamais faire passer une supposition pour un fait.
3. Comment présenter l'invariant C (« donnée insuffisante ») sans transformer le journal en liste d'erreurs anxiogène ?	Contrainte interne : la nutrition ne doit jamais devenir une source de stress supérieure au bénéfice qu'elle apporte.

8. Vérifiable

Le code réel des fonctions citées est dans le dossier d'audit déjà transmis (04_CODE_SOURCE.md, extrait verbatim et régénérable par `python3 tools/audit-nutrition.py`). Ce PDF est lui aussi généré : `python3 tools/gen_bug_reprise_pdf.py`.

Suite de tests complète au moment de la livraison : parcours 3339/3339, calculs 339/339, muscles 241/241, croisés 50/50, dates 9/9, données classées 0 trou. Déploiement vérifié (run #1027, 7 étapes en success).